

Fractional

Diogo Marto
Universidade de Aveiro
Aveiro, Portugal
diogo.marto@ua.pt

Abstract—We propose adaptive fractional gradient optimizers based on the ψ -Hilfer derivative, where the fractional order is defined as a function of the gradient norm. This formulation enables smooth interpolation between fractional dynamics for large gradients and classical updates near convergence. We introduce three variants—FRACGRADIENT, FRACGRADIENT V2, and FRACGRADIENT V2 ADAPTIVE—and evaluate them on MNIST, HappyFace, and CIFAR-10 across architectures including CNNs, ResNet50V2, and MobileNetV2. The results show that fractional methods improve convergence stability in challenging regimes and, in several cases, match or surpass SGD and Adam in accuracy. The adaptive variant further reduces sensitivity to learning rate initialization, offering practical robustness. Limitations include higher computational cost and sensitivity to the β parameter. Overall, adaptive fractional optimization emerges as a promising alternative to classical methods, bridging theoretical advances in fractional calculus with practical deep learning applications.

Index Terms—Fractional Derivatives, Numerical Methods, Optimization Algorithms, Neural Networks

I. INTRODUCTION

With the rapid development of deep learning, deep neural networks (DNNs) have achieved remarkable success in fields such as computer vision, natural language processing, and large language models. This progress is largely driven by innovations in neural architectures, while advances in optimization algorithms have been comparatively incremental. Despite the introduction of many new techniques, the training of DNNs still predominantly relies on gradient-based methods such as stochastic gradient descent (SGD) and its variants. While effective in practice, these optimizers often face challenges in highly non-convex and high-dimensional loss landscapes, where they may converge slowly, become trapped in suboptimal minima, or exhibit sensitivity to hyperparameters. This motivates the search for alternative optimization paradigms that can provide more flexible and robust convergence dynamics.

Fractional calculus has emerged as a promising framework for extending optimization beyond classical integer-order derivatives. By allowing derivatives of non-integer order, fractional gradient methods incorporate notions of non-locality and memory, which can enrich the learning dynamics of neural networks. Recent studies suggest that fractional derivatives can improve convergence behavior and generalization; however, most existing approaches employ a fixed fractional order throughout training. This rigidity limits their adaptability, as the optimal degree of “fractionality” may vary across different stages of the optimization process.

In this work, we address this limitation by developing an adaptive fractional-order optimization framework based on the ψ -Hilfer derivative. Specifically, we define the fractional order as a function of the gradient norm:

$$\alpha(N) = 1 - \frac{2}{\pi} \arctan(\beta N). \quad (1)$$

This formulation ensures that near convergence, when $\|\nabla J(\Theta)\|$ is small, $\alpha \rightarrow 1$, recovering the behavior of classical gradient descent. Conversely, when far from the optimum, $\alpha < 1$, enabling fractional-order dynamics that encourage more exploratory updates. In this way, the optimizer naturally adapts its behavior to the training stage without manual intervention.

Building on this principle, we propose three algorithms—FRACGRADIENT, FRACGRADIENT V2, and FRACGRADIENT V2 ADAPTIVE—which extend fractional optimization to both network-wide and per-layer adaptation, with additional learning rate adjustments in the adaptive variant. We implement these optimizers in modern deep learning frameworks and evaluate them across a range of neural architectures, including feedforward networks, convolutional neural networks (CNNs), ResNet, and MobileNet, on standard benchmarks such as MNIST, CIFAR-10, and a facial expression recognition dataset.

Contributions. (i) A ψ -Hilfer derivative-based adaptive fractional optimization framework; (ii) design and implementation of optimizers that adjust the fractional order as a function of the gradient norm; (iii) extensive experiments demonstrating competitive accuracy and improved convergence stability.

II. RELATED WORK

A. Classical Optimization Methods in Deep Learning

Optimization is central to the training of deep neural networks, and most practical algorithms are based on first-order gradient descent. Early methods such as stochastic gradient descent (SGD) and momentum [1], [2] remain widely used due to their simplicity and scalability. Improvements have primarily focused on accelerating convergence and improving stability through adaptive learning rates. Examples include AdaGrad [3], RMSProp [4], and Adam [5], which has become the default optimizer in many applications. More recent extensions such as AdamW [6], RAdam [7], and QHAdam [8] address weaknesses in regularization and variance adaptation. While these optimizers are effective, they remain grounded

in integer-order derivatives, which may limit their ability to model more complex optimization dynamics.

B. Fractional Calculus in Optimization

Fractional calculus generalizes differentiation and integration to non-integer orders, introducing properties such as non-locality and memory. These characteristics make fractional operators attractive for systems with long-range dependencies or irregular dynamics, and they have been applied in diverse domains including physics, biology, and signal processing [9], [10]. In the context of optimization, fractional gradient descent methods (FGDMs) extend classical gradient descent by replacing integer-order gradients with fractional-order ones. Pu et al. [11] first applied fractional gradients to quadratic functions, showing altered convergence behavior. Wei et al. [12] further studied convergence issues and proposed remedies such as truncating higher-order terms or adopting variable-order schemes. Subsequent works explored improved fractional gradient formulations [13], [14] and introduced variable-order strategies [15]. Despite these efforts, most studies remain limited to theoretical analyses or small-scale test problems, with relatively few applications to modern deep neural networks.

C. Variable-Order Fractional Gradient Methods

A growing line of research has investigated variable-order FGDMs, where the fractional order is not fixed but instead changes during training. Lou et al. [16] proposed a variable-order function for training backpropagation networks, while Chen et al. [15] designed a fractional-order scheduler for DNNs based on uncertainty theory. These works highlight the potential of variable-order methods, but their application to large-scale, modern architectures such as CNNs or ResNets remains underexplored. Moreover, most approaches do not directly link the fractional order to the optimization state, limiting their adaptivity.

D. Position of This Work

Our study builds upon this body of research by introducing a ψ -Hilfer derivative-based optimizer in which the fractional order is dynamically determined as a function of the gradient norm. Unlike fixed-order FGDMs, our method allows the fractional order to vary smoothly during training: approaching one near convergence and decreasing when the gradient norm is large. This mechanism provides a natural balance between classical gradient descent and fractional dynamics. Furthermore, we implement both network-wide and per-layer adaptive variants, and validate their performance on a range of neural architectures and datasets, moving beyond the small-scale settings that dominate prior FGDM studies.

III. PRELIMINARIES

This work developed a new neural network optimizer, called FRACGRADIENT, based on Algorithm 3 of [17].

VARIABLE ORDER FRACTIONAL GRADIENT DESCENT

The main distinction between this work and others in this area is that the fractional order α is chosen as a function of the norm of the gradient of the loss, $\|\nabla J(\Theta)\|$. The implemented algorithms support a family of functions; in this report we consider

$$\alpha(N) = 1 - \frac{2}{\pi} \arctan(\beta N), \quad (2)$$

where $N = \|\nabla J(\Theta)\|$. We infer that when $N \approx 0$ one has $\alpha(N) \approx 1$, and when $N \gg 0$ one has $\alpha(N) \approx 0$. That is, when $\|\nabla J(\Theta)\|$ is near 0 (after convergence), the fractional order is near 1 and we fall back to classical derivatives; when $\|\nabla J(\Theta)\| > 0$, the fractional order lies in $(0, 1)$.

The central idea is to allow greater flexibility in learning by adjusting the influence of the fractional exponent α , thereby controlling the intensity of the updates.

Algorithm 1

Algorithm 1 FracGradient

```

1: Inputs:
2:   Function:  $J(\Theta)$ 
3:   Fixed parameters:  $\alpha(\cdot)$ ,  $\beta$ ,  $\theta$ ,  $k$ 
4:   for  $i \leftarrow 1$  to  $k$  do
5:      $f \leftarrow \alpha(\|\nabla J(\Theta_{i-1})\|)$ 
6:      $\Theta_{i+1} \leftarrow \Theta_i - \theta \frac{\nabla J(\Theta_{i-1})}{\Gamma(2-f)} \|\Theta_i - \Theta_{i-1}\|^{1-f}$ 
7:   end for
```

Notes.

- $J(\Theta)$ is the loss function to minimize.
- Θ is a list of matrices with the neural network weights.
- $\alpha(\cdot)$ and β define the function controlling the fractional order.
- θ is the learning rate; k is the number of epochs.

In essence, this algorithm determines the fractional order f from (2). It applies the formula described in [17] to compute the fractional-order gradient and then updates the weights similarly to standard gradient descent but using the fractional gradient. The computation of $\nabla J(\Theta)$ is done via standard backpropagation.

Algorithm 2 FracGradient V2

```

1: Inputs:
2:   Function:  $J(\Theta)$ 
3:   Fixed parameters:  $\alpha(\cdot)$ ,  $\beta$ ,  $\theta$ ,  $k$ 
4:   for  $i \leftarrow 1$  to  $k$  do
5:     for  $n \leftarrow 1$  to  $\text{num\_layers}$  do
6:        $f_n \leftarrow \alpha(\|\nabla J(\Theta_{i-1}^{(n)})\|)$ 
7:        $\Theta_{i+1}^{(n)} \leftarrow \Theta_i^{(n)} - \theta \frac{\nabla J(\Theta_{i-1}^{(n)})}{\Gamma(2-f_n)} \|\Theta_i^{(n)} - \Theta_{i-1}^{(n)}\|^{1-f_n}$ 
8:     end for
9:   end for
```

The main change from Algorithm 1 to Algorithm 2 is that f_n is computed from the gradient norm of each layer $\Theta^{(n)}$ and

applied only to that layer, allowing each layer to adapt its own fractional order, whereas in the last algorithm, this order was fixed for the entire neural network.

Algorithm 3 FracGradient V2 Adaptive

```

1: Inputs:
2:   Function:  $J(\Theta)$ 
3:   Fixed parameters:  $\alpha(\cdot)$ ,  $\beta$ ,  $\theta_i$ ,  $k$ ,  $r_d$ ,  $r_i$ 
4:  $\theta \leftarrow \theta_i$ 
5: for  $i \leftarrow 1$  to  $k$  do
6:   for  $n \leftarrow 1$  to  $\text{num\_layers}$  do
7:      $f_n \leftarrow \alpha(\|\nabla J(\Theta_{i-1}^{(n)})\|)$ 
8:      $\Theta_{i+1}^{(n)} \leftarrow \Theta_i^{(n)} - \theta \cdot$ 
        $\frac{\nabla J(\Theta_{i-1}^{(n)})}{\Gamma(2-f_n)} \|\Theta_i^{(n)} - \Theta_{i-1}^{(n)}\|^{1-f_n}$ 
9:   end for
10:  if  $J(\Theta_i) > J(\Theta_{i-1})$  then
11:     $\theta \leftarrow \theta \cdot r_d$  ▷ decrease LR
12:  else
13:     $\theta \leftarrow \theta \cdot r_i$  ▷ increase LR
14:  end if
15: end for

```

The FracGradient V2 Adaptive algorithm is an extension of the last algorithm, where we update the learning rate θ based on whether in that iteration the cost function $J(\Theta_i)$ was lower than the previous iteration cost function $J(\Theta_{i-1})$. The learning-rate updates are controlled by r_d and r_i , which decrease or increase θ , respectively. The goal is to let the algorithm adapt θ to maximize convergence speed and reduce sensitivity to the initial choice θ_i .

IV. EXPERIMENTS

A. Structure of Experiments

To evaluate the performance of the FracGradient optimiser, we conducted three sets of experiments using neural network models for image classification tasks. Tests were performed using the MNIST [18], HappyFace [19], and CIFAR-10 [20] datasets with representative yet straightforward architectures. These datasets were chosen because they have different degrees of complexity that illustrate the behaviour of our algorithms in various scenarios.

The MNIST dataset contains grayscale images of handwritten digits (28×28). It includes 60,000 examples for training and 10,000 for testing, divided equally among 10 classes. All data was normalised to a float from 0 to 1, and the classes were converted to one-hot format.

The HappyFace dataset contains coloured images (64×64) of Happy and Non-Happy facial expressions, with 600 examples for training and 150 for testing. The class distribution is slightly unbalanced. All data was normalised to a float from 0 to 1, and the classes were converted to one-hot format.

CIFAR-10 consists of coloured images (32×32) distributed across 10 classes, with 50,000 images for training and 10,000 for testing, divided equally among 10 classes. All data was

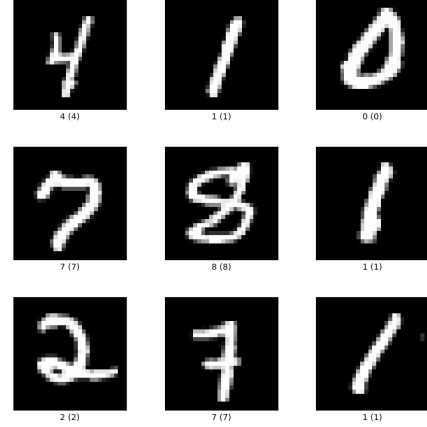


Fig. 1. Sample images from the MNIST dataset.

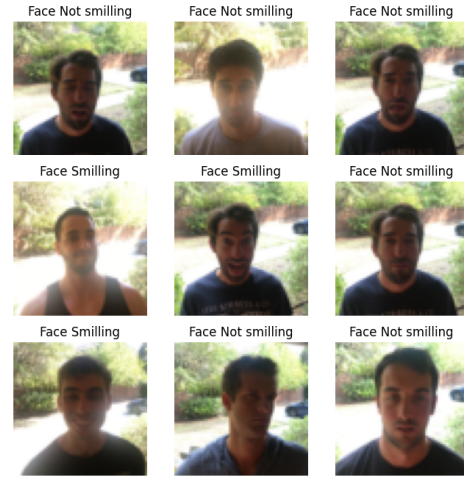


Fig. 2. Sample images from the HappyFace dataset.

normalised to a float from 0 to 1, and the classes were converted to one-hot format.

A total of 13 experiments were conducted across these datasets:

- For the MNIST dataset, three neural network architectures and a custom CNN were tested.
- For the HappyFace dataset, three neural network architectures and a custom CNN were tested.
- For the CIFAR-10 dataset, a simple CNN was tested with and without data augmentation, ResNet34V2 with data augmentation, and MobileNetV2 with data augmentation.

For the tests on the simple neural networks, grid search was used to find the best hyperparameters that minimized the final value of the loss function. For the CNNs and more complex models, the appropriate hyperparameters (namely the learning rate and β) were chosen manually for the SGD and FracOptimizerV2 optimisers.

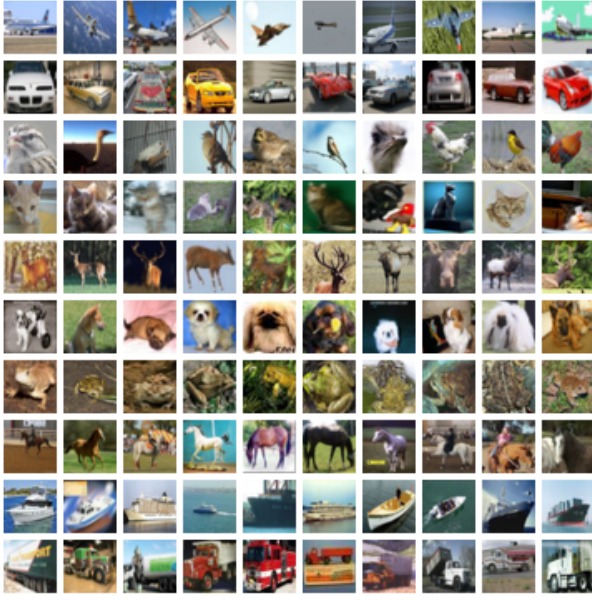


Fig. 3. Sample images from the CIFAR-10 dataset.

The comparison of the experiments focuses on the evolution of the cost function (loss) during training, time to finish all epochs, and the final accuracy obtained on the testing and training data using different optimisers. Accuracy is a suitable metric for these datasets since they are relatively balanced.

B. Testing Configuration

The tests were conducted in a local Python environment on Windows 11. The host machine running these tests had an AMD Ryzen 7 7800X3D CPU with 32 GB of RAM and an NVIDIA RTX 4080 Super GPU. For the experiments on the CIFAR-10 dataset, a WSL environment was used to run TensorFlow on the GPU.

C. MNIST 1 – 1 Hidden Layer of Size 25

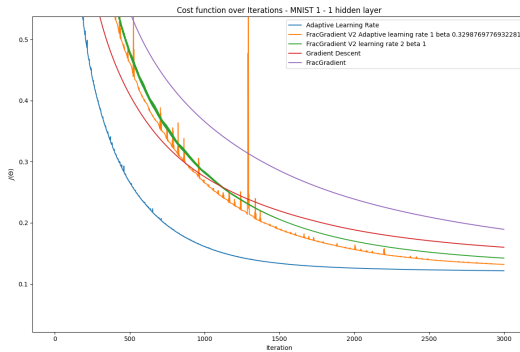


Fig. 4. Cost function $J(\Theta)$ over iterations for different optimisers for MNIST dataset experiment 1.

The plot in fig. 4 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ over 3000 iterations of the MNIST dataset with

an architecture consisting of a hidden layer of size 25. Five methods are compared: Gradient Descent, Adaptive Learning Rate, FracGradient, FracGradient V2, and FracGradient V2 Adaptive. The Adaptive Learning Rate is an enhancement of Gradient Descent that adjusts the step size dynamically during training.

Among the methods evaluated, the Adaptive Learning Rate approach demonstrated the fastest convergence and achieved the lowest final cost value. The Gradient Descent and FracGradient methods performed worse than their adaptive learning rate counterpart. FracGradient V2 showed improvement over the original FracGradient and Gradient Descent, but still lagged behind the adaptive methods. The FracGradient V2 Adaptive method achieved improved convergence rates compared to its non-adaptive counterparts; however, it displayed noticeable oscillations and instability, likely due to fluctuations in the adaptive learning rate.

TABLE I
RESULTS FOR MNIST 1 – 1 HIDDEN LAYER (25 UNITS).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive	1.00	0.93	18.69	0.122
Classic Optimizer	1.00	0.92	20.79	0.160
FracAdap	1.00	0.92	20.29	0.131
FracOptimizer V2	1.00	0.93	18.60	0.142
FracOptimizer	0.99	0.93	20.36	0.189

This performance trend is further supported by the results in table I, where the Adaptive Learning Rate optimiser achieved the lowest final loss (0.122). However, it is worth noting that all methods yielded comparable accuracy and training time.

D. MNIST 2 – 1 Hidden Layer of Size 6

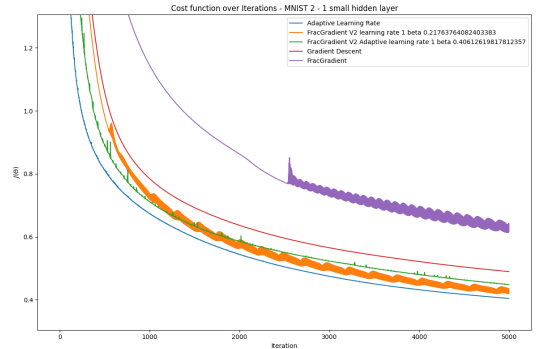


Fig. 5. Cost function $J(\Theta)$ over iterations for different optimisers for MNIST dataset experiment 2.

The plot in fig. 5 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ over 5000 iterations of the MNIST dataset with an architecture consisting of a hidden layer of size 6, compared to the previous experiment where the hidden layer size was 25. This was done to simulate harder learning conditions.

The results are similar to the previous experiment: the Adaptive Learning Rate optimiser achieved the best performance regarding the loss curve, but interestingly, FracGradient

V2 outperformed its adaptive counterpart, albeit with more instability.

TABLE II
RESULTS FOR MNIST 2 – 1 HIDDEN LAYER (6 UNITS).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive Learning Rate	0.92	0.86	22.62	0.404
Classic Optimizer	0.91	0.84	22.68	0.489
FracAdap	0.94	0.87	25.12	0.448
FracOptimizer V2	0.94	0.87	25.11	0.419
FracOptimizer	0.92	0.86	23.14	0.640

This performance trend is further supported by the results in table II, where the Adaptive Learning Rate optimizer achieved the lowest final loss (0.404). However, in terms of accuracy, both FracGradient V2 variants outperform the other optimizers.

E. MNIST 3 – 2 Hidden Layers of Size 32 and 16

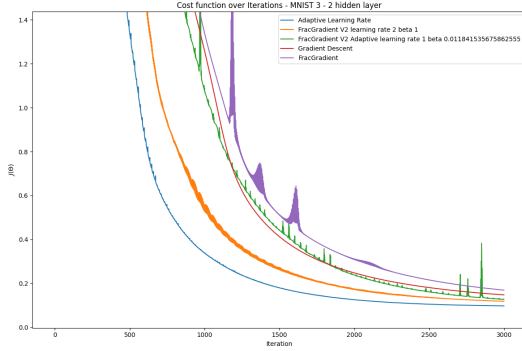


Fig. 6. Cost function $J(\Theta)$ over iterations for different optimisers for MNIST dataset experiment 3.

The plot in fig. 6 illustrates the performance of various gradient descent optimization algorithms in minimizing the cost function $J(\Theta)$ over 3000 iterations of the MNIST dataset with an architecture consisting of hidden layers of size 32 and 16. This was done to observe the behavior of the optimizer in a larger neural network.

The results are similar to those of the previous experiment, but the final loss values are closer across all optimizers because of the more complex architecture.

TABLE III
RESULTS FOR MNIST 3 – 2 HIDDEN LAYERS (32 AND 16 UNITS).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive Learning Rate	1.00	0.94	24.46	0.090
Classic Optimizer	1.00	0.94	25.03	0.147
FracAdap	1.00	0.93	26.00	0.127
FracOptimizer V2	1.00	0.92	26.39	0.118
FracOptimizer	0.99	0.92	30.58	0.160

This performance trend is further supported by the results in table III, where the Adaptive Learning Rate optimizer achieved the lowest final loss (0.090). In terms of accuracy, all optimizers are similar, but the non-fractional ones perform slightly better.

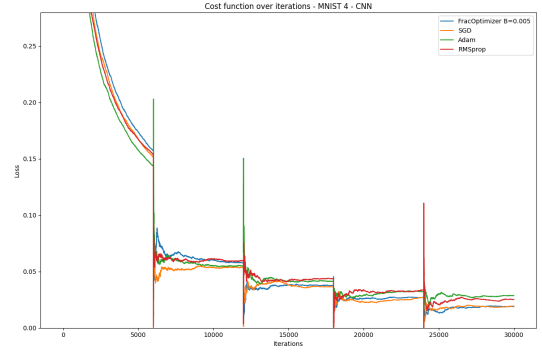


Fig. 7. Cost function $J(\Theta)$ over iterations for different optimisers for MNIST dataset experiment 4.

F. MNIST 4 – CNN

The plot in fig. 7 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the MNIST dataset over 30,000 iterations with batch size 10, split into 5 epochs of 6000 iterations each, using the architecture shown in table IV. This experiment was run on TensorFlow, and only the FracGradient V2 variant was tested in this configuration.

From the figure we observe that all optimisers converge smoothly. SGD and FracGradient V2 achieve the lowest final loss, while Adam shows the fastest initial convergence. The spikes around multiples of 6000 iterations are due to epoch changes, where the dataset is reshuffled.

TABLE IV
CNN ARCHITECTURE USED FOR MNIST EXPERIMENT 4.

Layer (type)	Output Shape	Param #
Conv2D (6 filters, 5×5)	(None, 28, 28, 6)	156
MaxPooling2D	(None, 14, 14, 6)	0
Conv2D (16 filters, 5×5)	(None, 10, 10, 16)	2,416
MaxPooling2D	(None, 5, 5, 16)	0
Conv2D (120 filters, 5×5)	(None, 1, 1, 120)	48,120
Flatten	(None, 120)	0
Dense (84 units)	(None, 84)	10,164
Dense (10 units)	(None, 10)	850
Total params		61,706 (241.04 KB)
Trainable params		61,706 (241.04 KB)

TABLE V
RESULTS FOR MNIST 4 – CNN.

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer V2	1.00	0.99	65.59	0.018
SGD	1.00	0.99	44.72	0.019
Adam	0.99	0.99	45.62	0.028
RMSProp	1.00	0.99	44.89	0.025

The accuracy of all optimizers is similar, as shown in table V. However, the FracOptimizer V2 took longer to train than the others. This could be due to a non-optimal implementation or the inherent cost of element-wise exponentiation required by the algorithm, which is slower than simple multiplication and addition.

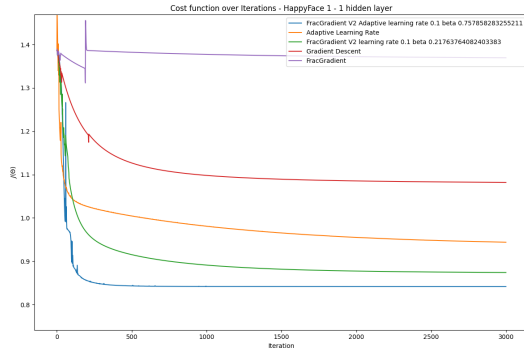


Fig. 8. Cost function $J(\Theta)$ over iterations for different optimisers for HappyFace dataset experiment 1.

G. HappyFace 1 – 1 Hidden Layer of Size 25

The plot in fig. 8 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the HappyFace dataset over 3000 iterations on a neural network with a hidden layer of size 25. Five optimisers were compared: Adaptive, Classic Optimizer, FracAdap, FracOptimizer V2, and FracOptimizer.

TABLE VI
RESULTS FOR HAPPYFACE 1 – 1 HIDDEN LAYER (25 UNITS).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive Learning Rate	0.77	0.67	65.04	0.943
Classic Optimizer	0.70	0.62	64.01	1.080
FracAdap	0.81	0.67	76.39	0.841
FracOptimizer V2	0.80	0.75	75.80	0.874
FracOptimizer	0.52	0.45	99.12	1.369

We can observe that FracOptimizer V2 with adaptive learning rate performed the best, followed by the regular FracOptimizer V2, both of which achieved a significant margin over the other optimisers. The FracOptimizer showed no improvement after the first few epochs and did not converge to a good local minimum.

From table VI, we can see that in terms of train and test accuracy, both FracOptimizer V2 variants obtained the best results. However, their training times were about 17% longer compared to Gradient Descent. Also, since this dataset has only two classes, the FracOptimizer performed worse than random guessing (50%) on the test dataset.

H. HappyFace 2 – 1 Hidden Layer of Size 64

The plot in fig. 9 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the HappyFace dataset over 3000 iterations on a neural network with a hidden layer of size 64. This was done to try to improve the accuracy compared to the previous experiment, where it was around 80% for the best optimisers.

We can observe that FracOptimizer V2 with adaptive learning rate performed the best, followed by the regular FracOptimizer V2, both of which achieved a significant margin over the other optimisers. The FracOptimizer V2 with adaptive

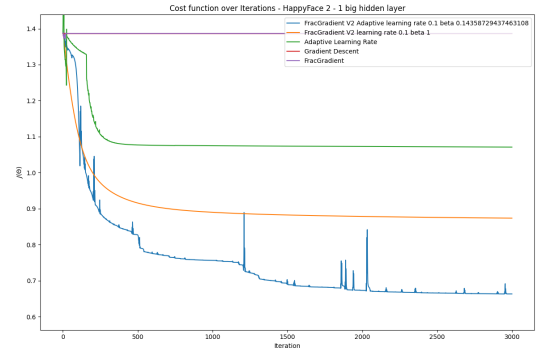


Fig. 9. Cost function $J(\Theta)$ over iterations for different optimisers for HappyFace dataset experiment 2.

learning rate shows some instability in the form of spikes. Both FracOptimizer and Gradient Descent showed very poor performance in this experiment.

TABLE VII
RESULTS FOR HAPPYFACE 2 – 1 HIDDEN LAYER (64 UNITS).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive Learning Rate	0.74	0.65	84.93	1.070
Classic Optimizer	0.50	0.55	80.99	1.386
FracAdap	0.85	0.79	110.58	0.663
FracOptimizer V2	0.78	0.67	110.22	0.870
FracOptimizer	0.50	0.55	164.48	1.386

From table VII, we can see that in terms of train and test accuracy, both FracOptimizer V2 variants obtained the best results, although the final accuracy was only slightly better compared to the previous experiment (0.85 vs. 0.81 on train and 0.79 vs. 0.75 on test). The trend that the FracOptimizers take more time to train was also more evident: in this experiment, they took about 37% more time to complete the 3000 iterations.

I. HappyFace 3 – 2 Hidden Layers of Size 64 and 32 (Sigmoid Activations)

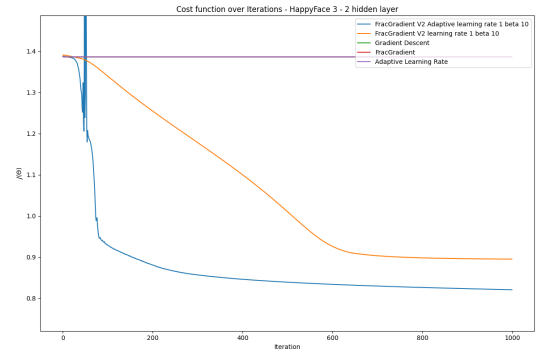


Fig. 10. Cost function $J(\Theta)$ over iterations for different optimisers for HappyFace dataset experiment 3.

The plot in fig. 10 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the HappyFace dataset over 1000

iterations on a neural network with hidden layers of size 64 and 32 but using sigmoid activation functions. This setup was intended to provide insight into optimiser behaviour under less-than-ideal learning conditions.

From fig. 10, we can see that only the FracOptimizer V2 variants were capable of learning; all other optimisers exhibit a flat cost curve, failing to reduce the loss.

TABLE VIII
RESULTS FOR HAPPYFACE 3 – 2 HIDDEN LAYERS (64 AND 32 UNITS, SIGMOID ACTIVATIONS).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive Learning Rate	0.50	0.56	26.40	1.386
Classic Optimizer	0.50	0.56	26.07	1.386
FracAdap	0.80	0.65	35.76	0.820
FracOptimizer V2	0.79	0.67	36.71	0.895
FracOptimizer	0.50	0.56	53.42	1.386

From table VIII, we see that in terms of train and test accuracy, both FracOptimizer V2 variants obtained the best results. The trend of FracOptimizers requiring more training time also persisted: in this experiment, they took about 34% longer to complete the 1000 iterations.

J. HappyFace 4 – 2 Hidden Layers of Size 64 and 32

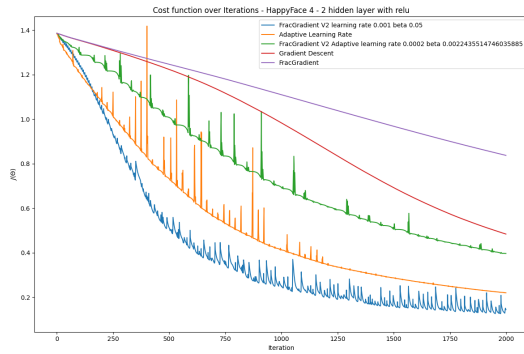


Fig. 11. Cost function $J(\Theta)$ over iterations for different optimisers for HappyFace dataset experiment 4.

The plot in fig. 11 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the HappyFace dataset over 2000 iterations on a neural network with hidden layers of size 64 and 32, using ReLU and sigmoid activation functions. This represents a good architecture for this dataset.

From the figure we observe a more balanced outcome, where all optimisers were capable of learning. The regular FracOptimizer V2 achieved the best final loss, but showed instability in the form of constant oscillations. FracOptimizer V2 with adaptive learning rate performed much worse than both the regular FracOptimizer V2 and the regular adaptive learning rate method. This may be due to the instability-driven increase in loss causing repeated reductions of the learning rate. Gradient Descent performed second worst (ahead of FracOptimizer) but exhibited the smoothest loss curve.

From table IX, we see that in terms of test accuracy all optimisers except FracOptimizer reached a similar score (0.95). In

TABLE IX
RESULTS FOR HAPPYFACE 4 – 2 HIDDEN LAYERS (64 AND 32 UNITS, RELU + SIGMOID).

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
Adaptive Learning Rate	0.99	0.95	51.47	0.219
Classic Optimizer	0.96	0.95	53.20	0.483
FracAdap	0.97	0.95	75.68	0.396
FracOptimizer V2	0.99	0.95	75.85	0.143
FracOptimizer	0.91	0.86	106.00	0.546

training accuracy, FracOptimizer V2 and the adaptive learning rate performed best (0.99). The trend of higher training times for FracOptimizers was again evident: here they took about 41% more time to complete the 2000 iterations.

K. HappyFace 5 – CNN

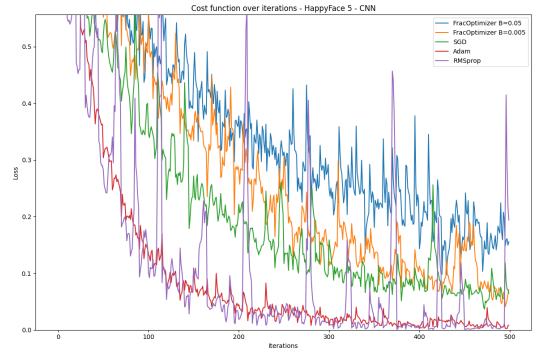


Fig. 12. Cost function $J(\Theta)$ over iterations for different optimisers for HappyFace dataset experiment 5.

The plot in fig. 12 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the HappyFace dataset over 500 iterations with batch size 128, split into 100 epochs of 5 iterations each, using the CNN architecture shown in table X. This experiment was run on TensorFlow, and only the FracGradient V2 variant was tested in this configuration.

From the figure we see that RMSProp and Adam achieved the lowest final loss, although RMSProp displayed significant spikes in its curve. SGD and FracOptimizer V2 with $\beta = 0.005$ showed similar loss curves, while FracOptimizer V2 with $\beta = 0.05$ performed the worst.

From figs. 13 and 14, we observe the fractional order of each layer over the 500 iterations. fig. 14 has values closer to 1 compared to fig. 13, as expected since the β parameter is smaller. Because the values of FracOptimizer V2 with $\beta = 0.005$ are near 1, its behaviour resembles SGD.

In table XI, we see that all optimisers except FracOptimizer V2 with $\beta = 0.05$ achieved similar accuracy. Training times were also similar across methods, in contrast with experiment MNIST 4, due to the smaller iteration count: here the optimiser ran 500 iterations in 19 seconds, compared to 30,000 iterations in 45 seconds in MNIST 4. Thus, the time overhead of the fractional methods was not significant in this experiment.

TABLE X
CNN ARCHITECTURE USED FOR HAPPYFACE EXPERIMENT 5.

Layer (type)	Output Shape	Param #
BatchNormalization	(None, 64, 64, 3)	12
Conv2D (32 filters)	(None, 64, 64, 32)	1,568
MaxPooling2D	(None, 16, 16, 32)	0
Dropout	(None, 16, 16, 32)	0
Conv2D (64 filters)	(None, 16, 16, 64)	32,832
MaxPooling2D	(None, 4, 4, 64)	0
Dropout	(None, 4, 4, 64)	0
Flatten	(None, 1024)	0
Dense (256 units)	(None, 256)	262,400
Dropout	(None, 256)	0
Dense (2 units)	(None, 2)	514
Total params		297,326 (1.13 MB)
Trainable params		297,320 (1.13 MB)

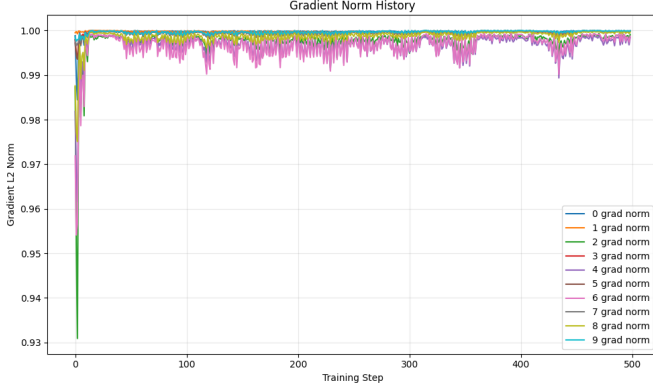


Fig. 13. Fractional order $\alpha(\|\nabla J(\Theta^{(n)})\|)$ per layer over iterations for HappyFace 5 with FracOptimizer V2, $\beta = 0.05$.

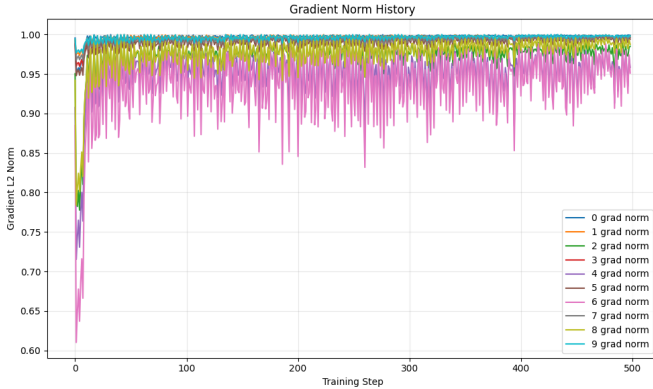


Fig. 14. Fractional order $\alpha(\|\nabla J(\Theta^{(n)})\|)$ per layer over iterations for HappyFace 5 with FracOptimizer V2, $\beta = 0.005$.

TABLE XI
RESULTS FOR HAPPYFACE 5 – CNN.

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer V2 ($\beta = 0.005$)	0.99	0.97	18.72	0.070
FracOptimizer V2 ($\beta = 0.05$)	0.90	0.82	19.24	0.155
SGD	0.99	0.95	18.90	0.064
Adam	1.00	0.97	19.05	0.008

L. CIFAR-10 1 – CNN without Data Augmentation

The plot in fig. 15 illustrates the performance of various gradient descent optimisation algorithms in minimising the

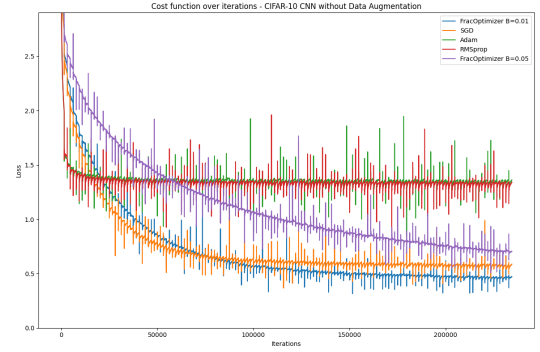


Fig. 15. Cost function $J(\Theta)$ over iterations for different optimisers for CIFAR-10 dataset (CNN without data augmentation).

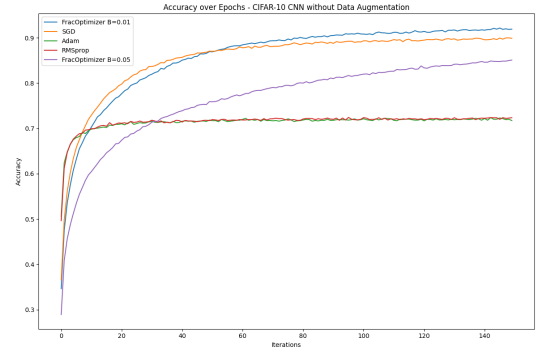


Fig. 16. Cost function $J(\Theta)$ over epochs for different optimisers for CIFAR-10 dataset (CNN without data augmentation).

cost function $J(\Theta)$ of the CIFAR-10 dataset over 234,450 iterations with batch size 32, split into 150 epochs of 1563 iterations each, using the CNN architecture shown in table XII, without data augmentation. This experiment was run on TensorFlow, and only the FracGradient V2 variant was tested in this configuration.

From the figure, we see that FracOptimizer V2 with $\beta = 0.01$ achieved the best final loss, while increasing β to 0.05 significantly worsened performance. Adam and RMSProp did not generalise well, likely becoming stuck in sharp minima. SGD initially converged faster than FracOptimizer V2 with $\beta = 0.01$, but ultimately FracOptimizer V2 with $\beta = 0.01$ achieved a lower score.

The spikes visible in fig. 15 correspond to reshuffling between epochs. Viewing the loss by epoch in fig. 16 shows smoother convergence.

From table XIII, we see that the train accuracy matched the loss behaviour. Despite having a lower train accuracy, FracOptimizer V2 with $\beta = 0.05$ slightly outperformed SGD on test accuracy (0.80 vs 0.79). In terms of training time, the FracOptimizer V2 variants were 27% ($\beta = 0.01$) and 37% ($\beta = 0.05$) slower than SGD.

M. CIFAR-10 2 – CNN with Data Augmentation

The plot in fig. 17 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost

TABLE XII
CNN ARCHITECTURE USED FOR CIFAR-10 EXPERIMENT 1 (WITHOUT DATA AUGMENTATION).

Layer (type)	Output Shape	Param #
BatchNormalization	(None, 32, 32, 3)	12
Conv2D (32 filters)	(None, 32, 32, 32)	896
MaxPooling2D	(None, 16, 16, 32)	0
Dropout	(None, 16, 16, 32)	0
Conv2D (64 filters)	(None, 16, 16, 64)	18,496
MaxPooling2D	(None, 8, 8, 64)	0
Dropout	(None, 8, 8, 64)	0
Conv2D (128 filters)	(None, 8, 8, 128)	73,856
MaxPooling2D	(None, 4, 4, 128)	0
Dropout	(None, 4, 4, 128)	0
Conv2D (256 filters)	(None, 4, 4, 256)	295,168
MaxPooling2D	(None, 2, 2, 256)	0
Dropout	(None, 2, 2, 256)	0
Flatten	(None, 1024)	0
Dense (512 units)	(None, 512)	524,800
Dropout	(None, 512)	0
Dense (10 units)	(None, 10)	5,130
Total params		918,358 (3.50 MB)
Trainable params		918,352 (3.50 MB)

TABLE XIII
RESULTS FOR CIFAR-10 1 – CNN WITHOUT DATA AUGMENTATION.

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer V2 ($\beta = 0.01$)	1.00	0.80	2084.92	0.471
FracOptimizer V2 ($\beta = 0.05$)	0.93	0.80	2254.30	0.700
SGD	0.98	0.79	1644.04	0.586
Adam	0.77	0.73	1818.85	1.361
RMSProp	0.78	0.74	1778.05	1.340

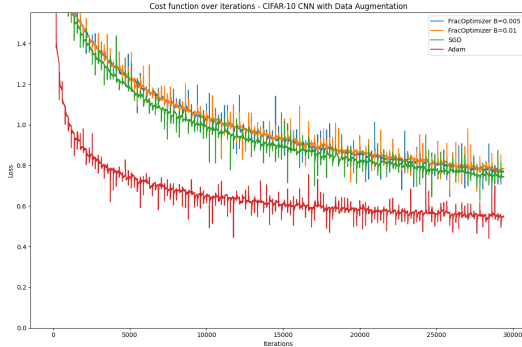


Fig. 17. Cost function $J(\Theta)$ over iterations for different optimisers for CIFAR-10 dataset (CNN with data augmentation).

function $J(\Theta)$ of the CIFAR-10 dataset over 29,400 iterations with batch size 256, split into 150 epochs of 196 iterations each, using the CNN architecture shown in table XII, but with data augmentation (parameters in fig. 18). This experiment was run on TensorFlow, and only the FracGradient V2 variant was tested in this configuration. Data augmentation was introduced to combat the overfitting observed in the previous experiment.

From the figure, we see that Adam performed best in this scenario, while all other optimisers exhibited similar loss curves. The improvement of Adam compared to the previous experiment may be due to data augmentation preventing it from getting stuck in sharp minima.

From table XIV, we see that all optimisers achieved similar training and testing accuracies, despite differences in the loss

- rotation range: 10
- width shift range: 0.15
- height shift range: 0.15
- horizontal flip: true
- zoom range: 0.1
- channel shift range: 0.12

Fig. 18. Parameters of data augmentation for CIFAR-10 experiment 2.

TABLE XIV
RESULTS FOR CIFAR-10 2 – CNN WITH DATA AUGMENTATION.

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer V2 ($\beta = 0.005$)	0.78	0.76	2546.36	0.769
FracOptimizer V2 ($\beta = 0.01$)	0.78	0.76	2541.71	0.779
SGD	0.79	0.77	2003.08	0.559
Adam	0.79	0.77	2005.00	0.840

curve. The trend that the FracOptimizers take longer to train remained: here, they were about 25% slower than SGD.

N. CIFAR-10 3 – ResNet50V2

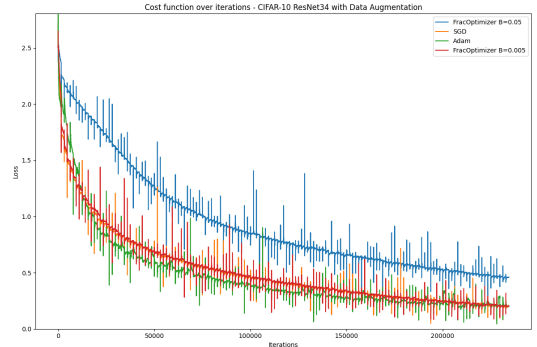


Fig. 19. Cost function $J(\Theta)$ over iterations for different optimisers for CIFAR-10 dataset (ResNet50V2 with data augmentation).

The plot in fig. 19 illustrates the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the CIFAR-10 dataset over 234,450 iterations with batch size 32, split into 150 epochs of 1563 iterations each, using the ResNet50V2 architecture and data augmentation parameters listed in fig. 18. This experiment was run on TensorFlow, and only the FracGradient V2 variant was tested in this configuration. The ResNet50V2 represents a significantly more complex model compared to the CNN used in the first CIFAR-10 experiments.

From the figure we see that all optimisers performed similarly, except for FracOptimizer V2 with $\beta = 0.05$, which performed worse. For FracOptimizer V2 with $\beta = 0.005$, all fractional-order gradients across layers were close to 1, meaning its behaviour closely resembled SGD.

From table XV, we see that all optimisers except FracOptimizer V2 with $\beta = 0.05$ achieved similar training accuracy, with Adam obtaining the highest test accuracy. The trend of FracOptimizers being slower remained: in this experiment, they were about 30% slower than SGD. It is also worth noting

TABLE XV
RESULTS FOR CIFAR-10 3 – RESNET50V2 WITH DATA AUGMENTATION.

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer V2 ($\beta = 0.005$)	0.96	0.80	31048.70	0.206
FracOptimizer V2 ($\beta = 0.05$)	0.92	0.76	31345.67	0.459
SGD	0.96	0.82	24865.80	0.207
Adam	0.95	0.85	27583.33	0.197

that all optimisers showed signs of overfitting, with training accuracy near 100%.

O. CIFAR-10 4 – MobileNetV2

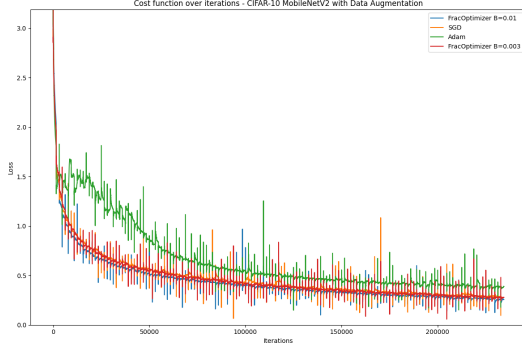


Fig. 20. Cost function $J(\Theta)$ over iterations for different optimisers for CIFAR-10 dataset (MobileNetV2 with data augmentation).

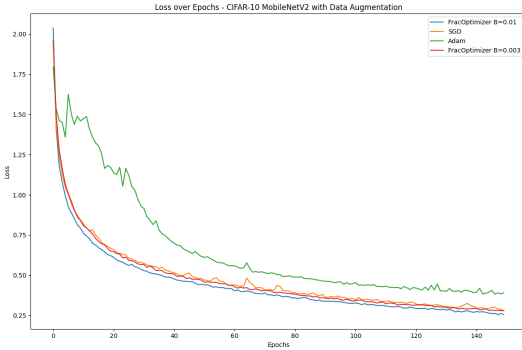


Fig. 21. Cost function $J(\Theta)$ over epochs for different optimisers for CIFAR-10 dataset (MobileNetV2 with data augmentation).

The plots in figs. 20 and 21 illustrate the performance of various gradient descent optimisation algorithms in minimising the cost function $J(\Theta)$ of the CIFAR-10 dataset over 234,450 iterations with batch size 32, split into 150 epochs of 1563 iterations each, using the MobileNetV2 architecture and the augmentation parameters in fig. 18. This experiment was run on TensorFlow, and only the FracGradient V2 variant was tested in this configuration.

From fig. 20, we see that all optimisers except Adam produced similar loss curves, with Adam showing worse behaviour. In fig. 21, FracOptimizer V2 with $\beta = 0.01$ achieved a slightly lower loss value than SGD. The fractional order chosen by the optimiser was significantly different from 1, as shown in fig. 22, contrasting with the previous experiment

where β was near 0 and the fractional order stayed close to 1, effectively mimicking SGD.

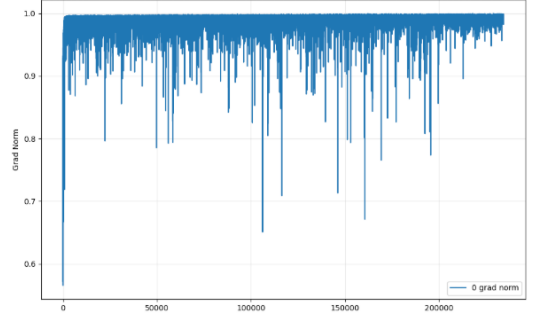


Fig. 22. Fractional order $\alpha(\|\nabla J(\Theta^{(n)})\|)$ in layer 0 over iterations for CIFAR-10 experiment 4 with FracOptimizer V2, $\beta = 0.01$.

TABLE XVI
RESULTS FOR CIFAR-10 4 – MOBILENETV2 WITH DATA AUGMENTATION.

Optimizer	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer V2 ($\beta = 0.003$)	0.90	0.83	15552.52	0.279
FracOptimizer V2 ($\beta = 0.01$)	0.94	0.85	13260.95	0.255
SGD	0.92	0.85	10730.10	0.279
Adam	0.88	0.83	11711.73	0.390

From table XVI, we see that FracOptimizer V2 with $\beta = 0.01$ achieved the highest training accuracy (0.94) and tied with SGD on test accuracy (0.85). The trend of FracOptimizers being slower remained: in this experiment, they were about 23% slower than SGD.

V. DISCUSSION

Our experimental results across MNIST, HappyFace, and CIFAR-10 provide a nuanced view of the potential and current limitations of adaptive fractional optimisation. Broadly, the empirical findings confirm the central design of our method: by adapting the fractional order $\alpha(N)$ as a function of the gradient norm, the optimiser interpolates between fractional dynamics when gradients are large and classical updates near convergence. This mechanism was consistently reflected in training curves, where FracGradient V2 and its adaptive variant displayed smoother convergence under challenging conditions, while reverting to SGD-like stability in later stages.

a) *Comparison with classical baselines.*: Against SGD, Adam, and RMSProp, our methods exhibited competitive or superior generalisation on several tasks. For example, on HappyFace with shallow networks, FracGradient V2 and FracGradient V2 Adaptive achieved higher test accuracy than SGD and adaptive learning rate, highlighting their robustness in low-data, noisy settings. On CIFAR-10, the fractional optimiser with carefully chosen β matched or slightly outperformed SGD on MobileNetV2, though Adam remained stronger in scenarios where data augmentation improved its stability. Importantly, fractional methods rarely underperformed dramatically; the main exception was when β was too large, which led to overly fractional behaviour and degraded convergence (e.g., FracOptimizer V2 with $\beta = 0.05$ on ResNet50V2).

b) *Stability and convergence.*: Fractional dynamics provided benefits, particularly in difficult learning setups. On HappyFace with sigmoid activations, where classical optimisers stalled, only the fractional variants were able to make progress. Similarly, in MNIST experiments with small hidden layers, the fractional optimisers achieved higher accuracy despite oscillatory loss curves, suggesting that adaptive fractional updates can help escape poor local minima. However, instability was also observed: in some cases, the adaptive learning rate combined with fractional updates amplified oscillations, pointing to an interaction between $\alpha(N)$ and step-size adaptation.

c) *Mechanism and theoretical link.*: These behaviours align with the theoretical intuition of fractional order methods. When gradients are large, $\alpha(N) < 1$ dampens aggressive updates, preventing divergence; when gradients diminish, $\alpha(N) \rightarrow 1$ recovers the stability of classical descent. The empirical evidence supports this interpretation: in MobileNetV2 training, layers with higher gradient norms indeed adopted smaller α , yielding exploratory dynamics early in training, before gradually approaching SGD-like updates. This adaptive balance addresses convergence and stability issues noted in earlier FGDM studies, where a fixed fractional order often led either to instability or to negligible benefits.

d) *Practical implications.*: The consistency of results across CNNs, ResNets, and MobileNets indicates that the method generalises across architectures. Nonetheless, the longer training times compared to SGD and Adam highlight the need for more efficient implementations, especially for large-scale settings.

e) *Limitations.*: Our study also exposes several limitations. Computationally, the ψ -Hilfer fractional derivative introduces element-wise exponentiation, making training slower (often 20–40% slower than SGD). Sensitivity to β remains an issue: small β values reduce the method to SGD-like behaviour, while large values can harm convergence, implying a need for principled scheduling or automatic tuning of β . Empirically, our evaluations are restricted to medium-scale datasets; extensions to ImageNet or transformer architectures are necessary to fully validate scalability.

Taken together, the experiments show that adaptive fractional optimisation is not a universal replacement for SGD or Adam, but offers distinct advantages in robustness and convergence under challenging conditions. By combining the flexibility of fractional dynamics with practical adaptivity, our framework bridges the gap between prior theoretical FGDM work and applicability in modern deep learning.

VI. CONCLUSION

In this work, we introduced a family of adaptive fractional gradient optimisers based on the ψ -Hilfer derivative, designed to dynamically interpolate between fractional and classical updates according to the gradient norm. Through extensive experiments on MNIST, HappyFace, and CIFAR-10 with diverse architectures ranging from simple feedforward networks to ResNet50V2 and MobileNetV2, we demonstrated that these

optimisers offer competitive accuracy, smoother training in challenging setups, and greater robustness compared to classical baselines such as SGD and Adam.

The empirical results highlight one key strength: fractional dynamics improved convergence stability in difficult training regimes, such as small networks or non-ideal activations, where traditional optimisers often stalled.

At the same time, our findings reveal important limitations. Training time was consistently higher than for SGD and Adam, reflecting the computational cost of fractional derivatives. Performance was sensitive to the choice of β , with poorly chosen values leading to worse loss curves. Moreover, our evaluation was restricted to medium-scale datasets, leaving open questions about scalability to large datasets and transformer-based architectures.

Taken together, our study suggests that adaptive fractional order optimisation is not yet a universal replacement for existing methods but provides a valuable complement. Future work should focus on more efficient implementations, automated β -scheduling, and validation on large-scale benchmarks.

REFERENCES

- [1] L. Bottou, “Large-scale machine learning with stochastic gradient descent,” in *Proceedings of COMPSTAT*. Springer, 2010, pp. 177–186.
- [2] L. Bottou and O. Bousquet, “The tradeoffs of large scale learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 20, 2007.
- [3] J. Duchi, E. Hazan, and Y. Singer, “Adaptive subgradient methods for online learning and stochastic optimization,” in *Conference on Learning Theory (COLT)*, 2011.
- [4] G. Hinton, “Lecture 6.5—rmsprop: Divide the gradient by a running average of its recent magnitude,” 2012, coursera: Neural Networks for Machine Learning.
- [5] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” *International Conference on Learning Representations (ICLR)*, 2015.
- [6] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [7] L. Liu, H. Jiang, P. He, W. Chen, X. Liu, J. Gao, and J. Han, “On the variance of the adaptive learning rate and beyond,” in *International Conference on Learning Representations (ICLR)*, 2020.
- [8] J. Ma and D. Yarats, “Quasi-hyperbolic momentum and adam for deep learning,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [9] R. L. Magin, M. D. Ortigueira, I. Podlubny, and J. J. Trujillo, “Fractional calculus in biomedical engineering,” *Mathematical Problems in Engineering*, vol. 2011, 2011.
- [10] S. Das and I. Pan, “Signal processing using fractional calculus,” *Signal Processing*, vol. 91, no. 3, pp. 386–404, 2011.
- [11] Y. Pu, Z. Ding, and T. Huang, “Fractional-order gradient methods,” *Neurocomputing*, vol. 154, pp. 235–244, 2015.
- [12] Y. Wei, X. Chen, and Y. Sun, “Convergence analysis of fractional-order gradient methods,” *Applied Mathematics and Computation*, vol. 374, p. 125061, 2020.
- [13] Y. Wang and S. Yan, “Fractional order gradient descent method and its application in neural networks,” *Neurocomputing*, vol. 365, pp. 112–121, 2019.
- [14] J. Liu, Z. Xu, and W. Wang, “Fractional-order gradient descent method for quadratic objective functions,” *Journal of Computational and Applied Mathematics*, vol. 389, p. 113342, 2021.
- [15] Z. Chen, L. Xu, and F. Li, “A variable-order fractional gradient scheduler for training deep neural networks,” *Information Sciences*, vol. 646, p. 119394, 2024.
- [16] W. Lou, W. Gao, X. Han, and Y. Zhang, “Variable order fractional gradient descent method and its application in neural networks optimization,” in *2022 China Control and Decision Conference (CCDC)*, 2022. [Online]. Available: <https://ieeexplore.ieee.org/document/10033456>

- [17] N. Vieira, M. M. Rodrigues, and M. Ferreira, "Fractional gradient methods via ψ -Hilfer derivative," *Fractal and Fractional*, vol. 7, no. 3, p. 275, 2023.
- [18] Y. LeCun, C. Cortes, and C. J. Burges, "The MNIST Database of Handwritten Digits," Website, 1998, accessed: 22 June 2025. [Online]. Available: <http://yann.lecun.com/exdb/mnist/>
- [19] A. Motwani, "HappyFace Dataset," Kaggle dataset, 2024, accessed: 22 June 2025. [Online]. Available: <https://www.kaggle.com/datasets/ashishmotwani/happyface>
- [20] A. Krizhevsky, "CIFAR-10 Dataset: Learning Multiple Layers of Features from Tiny Images," Tech. Rep., 2009, accessed: 22 June 2025. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>

APPENDIX
APPENDIX: EXPERIMENTAL RESULTS

TABLE XVII: Summary of optimiser performance across all experiments.

Experiment / Optimiser	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
MNIST 1 – 1 Hidden Layer (25 units)				
Adaptive	1.00	0.93	18.69	0.121
Classic Optimizer	1.00	0.92	20.79	0.160
FracAdap	1.00	0.92	20.29	0.131
FracOptimizer V2	1.00	0.93	18.60	0.142
FracOptimizer	0.99	0.93	20.36	0.189
MNIST 2 – 1 Hidden Layer (6 units)				
Adaptive	0.92	0.86	22.62	0.404
Classic Optimizer	0.91	0.84	22.68	0.489
FracAdap	0.94	0.87	25.12	0.448
FracOptimizer V2	0.94	0.87	25.11	0.419
FracOptimizer	0.92	0.86	23.14	0.640
MNIST 3 – 2 Hidden Layers (32, 16 units)				
Adaptive	1.00	0.94	24.46	0.090
Classic Optimizer	1.00	0.94	25.03	0.147
FracAdap	1.00	0.93	26.00	0.127
FracOptimizer V2	1.00	0.92	26.39	0.118
FracOptimizer	0.99	0.92	30.58	0.160
MNIST 4 – CNN				
FracOptimizer V2	1.00	0.99	65.59	0.018
SGD	1.00	0.99	44.72	0.019
Adam	0.99	0.99	45.62	0.028
RMSProp	1.00	0.99	44.89	0.025
HappyFace 1 – 1 Hidden Layer (25 units)				
Adaptive	0.77	0.67	65.04	0.943
Classic Optimizer	0.70	0.62	64.01	1.080
FracAdap	0.81	0.67	76.39	0.841
FracOptimizer V2	0.80	0.75	75.80	0.874
FracOptimizer	0.52	0.45	99.12	1.369
HappyFace 2 – 1 Hidden Layer (64 units)				
Adaptive	0.74	0.65	84.93	1.070
Classic Optimizer	0.50	0.55	80.99	1.386
FracAdap	0.85	0.79	110.58	0.663
FracOptimizer V2	0.78	0.67	110.22	0.870
FracOptimizer	0.50	0.55	164.48	1.386
HappyFace 3 – 2 Hidden Layers (64, 32 units, Sigmoid)				
Adaptive	0.50	0.56	26.40	1.386
Classic Optimizer	0.50	0.56	26.07	1.386
FracAdap	0.80	0.65	35.76	0.820
FracOptimizer V2	0.79	0.67	36.71	0.895

Experiment / Optimiser	Train Acc (%)	Test Acc (%)	Time (s)	Final Loss
FracOptimizer	0.50	0.56	53.42	1.386
HappyFace 4 – 2 Hidden Layers (64, 32 units, ReLU+Sigmoid)				
Adaptive	0.99	0.95	51.47	0.219
Classic Optimizer	0.96	0.95	53.20	0.483
FracAdap	0.97	0.95	75.68	0.396
FracOptimizer V2	0.99	0.95	75.85	0.143
FracOptimizer	0.91	0.86	106.00	0.546
HappyFace 5 – CNN				
FracOptimizer V2 ($\beta = 0.005$)	0.99	0.97	18.72	0.070
FracOptimizer V2 ($\beta = 0.05$)	0.90	0.82	19.24	0.155
SGD	0.99	0.95	18.90	0.064
Adam	1.00	0.97	19.05	0.008
CIFAR-10 1 – CNN (No Augmentation)				
FracOptimizer V2 ($\beta = 0.01$)	1.00	0.80	2084.92	0.471
FracOptimizer V2 ($\beta = 0.05$)	0.93	0.80	2254.30	0.700
SGD	0.98	0.79	1644.04	0.586
Adam	0.77	0.73	1818.85	1.361
RMSProp	0.78	0.74	1778.05	1.340
CIFAR-10 2 – CNN (With Augmentation)				
FracOptimizer V2 ($\beta = 0.005$)	0.78	0.76	2546.36	0.769
FracOptimizer V2 ($\beta = 0.01$)	0.78	0.76	2541.71	0.779
SGD	0.79	0.77	2003.08	0.559
Adam	0.79	0.77	2005.00	0.840
CIFAR-10 3 – ResNet50V2 (With Augmentation)				
FracOptimizer V2 ($\beta = 0.005$)	0.96	0.80	31048.70	0.206
FracOptimizer V2 ($\beta = 0.05$)	0.92	0.76	31345.67	0.459
SGD	0.96	0.82	24865.80	0.207
Adam	0.95	0.85	27583.33	0.197
CIFAR-10 4 – MobileNetV2 (With Augmentation)				
FracOptimizer V2 ($\beta = 0.003$)	0.90	0.83	15552.52	0.279
FracOptimizer V2 ($\beta = 0.01$)	0.94	0.85	13260.95	0.255
SGD	0.92	0.85	10730.10	0.279
Adam	0.88	0.83	11711.73	0.390